

AI/ML Internship – Day 60 Notes & Tasks

Module 8: Building a Menu-Driven Chatbot using Python

Part 1: Recap of Day 59

Yesterday we learned:

-  Rule-Based Chatbots
-  Using if-elif-else
-  Dictionary-based Chatbots
-  Keyword Matching

Today we will improve our chatbot by creating a **Menu-Driven Chatbot** that gives users options to choose from.

Part 2: What is a Menu-Driven Chatbot?

Definition

A Menu-Driven Chatbot is a chatbot that provides a list of predefined options (menu) for users to choose from instead of typing free-form questions.

Simple Explanation

Instead of asking users to type anything, the chatbot displays choices like:

 Welcome!

1. Course Fee
2. Course Duration
3. Contact Number
4. Location
5. Exit

Choose an option:

The chatbot responds based on the selected option.

Part 3: Why Use a Menu-Driven Chatbot?

Advantages

- ✓ Easy for beginners
 - ✓ No spelling mistakes
 - ✓ Faster responses
 - ✓ User-friendly
 - ✓ Less chance of invalid input
-

Part 4: Real-World Examples

ATM Machine

1. Check Balance
 2. Withdraw Money
 3. Deposit Money
 4. Exit
-

Restaurant Ordering System

1. Pizza
 2. Burger
 3. Juice
 4. Billing
-

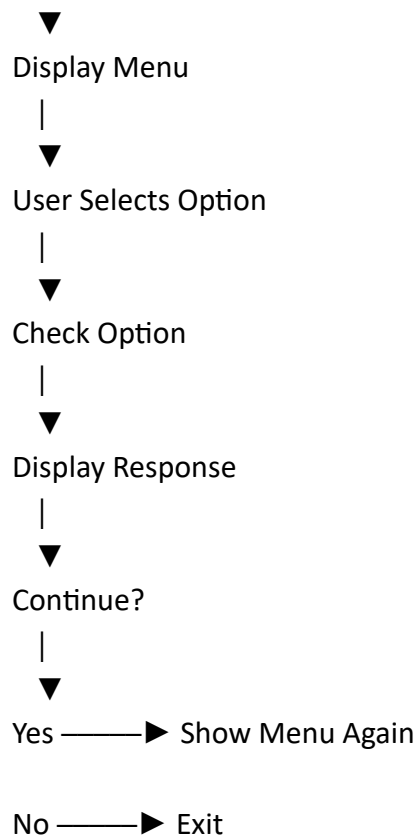
College Enquiry Bot

1. Fees
 2. Duration
 3. Eligibility
 4. Contact
 5. Exit
-

Part 5: Chatbot Workflow

Start

|



🔴 Part 6: Simple Menu Program

while True:

```
print("\n===== MENU =====")
print("1. Fees")
print("2. Duration")
print("3. Exit")
```

```
choice = input("Enter your choice: ")
```

```
if choice == "1":
    print("Course Fee: ₹15,000")
```

```
elif choice == "2":
    print("Duration: 3 Months")
```

```
elif choice == "3":
    print("Thank You!")
    break
```

```
else:
    print("Invalid Choice")
```

Part 7: Complete College Enquiry Chatbot

while True:

```
    print("\n=====")
    print("🎓 College Enquiry Chatbot")
    print("=====")
    print("1. Course Fee")
    print("2. Course Duration")
    print("3. Eligibility")
    print("4. College Location")
    print("5. Contact Number")
    print("6. Exit")
```

```
    choice = input("\nEnter your choice: ")
```

```
    if choice == "1":
        print("Course Fee: ₹15,000")
```

```
    elif choice == "2":
        print("Duration: 3 Months")
```

```
    elif choice == "3":
        print("Eligibility: Plus Two Pass")
```

```
    elif choice == "4":
        print("Location: Kochi")
```

```
    elif choice == "5":
        print("Phone: 9876543210")
```

```
    elif choice == "6":
        print("Thank You for Visiting!")
        break
```

```
    else:
        print("Invalid Choice! Please try again.")
```

Part 8: Understanding the Code

while True

Keeps the chatbot running until the user exits.

print()

Displays the chatbot menu.

input()

Reads the user's menu choice.

if-elif-else

Checks which option the user selected.

break

Stops the chatbot when the user selects Exit.

Part 9: Improving the Chatbot Using Dictionary

Instead of writing many if-elif statements, we can store menu items in a dictionary.

```
responses = {  
    "1": "Course Fee: ₹15,000",  
    "2": "Duration: 3 Months",  
    "3": "Eligibility: Plus Two Pass",  
    "4": "Location: Kochi",  
    "5": "Phone: 9876543210"  
}
```

while True:

```
    print("\n===== College Chatbot =====")  
    print("1. Fees")  
    print("2. Duration")
```

```
print("3. Eligibility")
print("4. Location")
print("5. Contact")
print("6. Exit")

choice = input("Enter your choice: ")

if choice == "6":
    print("Goodbye!")
    break

print(responses.get(choice, "Invalid Choice"))
```

Part 10: Adding More Features

You can add options like:

7. Hostel Information

8. Admission Process

9. Scholarship Details

10. Placement Information

This makes the chatbot more useful.

Part 11: Error Handling

Suppose the user enters:

20

or

abc

The chatbot should not crash.

Instead, display:

Invalid Choice!

Please select a valid option.

This improves user experience.

Part 12: Real-World Applications

College Admission Bot

Course information

Admission process

Fee details

Hospital Chatbot

Doctor availability

Appointment booking

Emergency contacts

Banking Chatbot

Balance enquiry

Loan details

Card services

Restaurant Chatbot

Food menu

Offers

Table booking

Hotel Chatbot

Room booking

Facilities

Check-in details

Part 13: Advantages

- ✓ Beginner-friendly
 - ✓ Easy to build
 - ✓ Fast response
 - ✓ Suitable for FAQs
 - ✓ No AI training required
-

Part 14: Limitations

- ✗ Users must choose from the menu.
 - ✗ Cannot answer unexpected questions.
 - ✗ No understanding of natural language.
 - ✗ No learning capability.
-

Part 15: Difference Between Menu-Driven and Rule-Based Chatbots

Rule-Based Chatbot	Menu-Driven Chatbot
User types questions	User selects options
May have spelling issues	No spelling issues
Uses text matching	Uses menu selection
More flexible	Easier for beginners
Better for simple conversations	Better for structured information

Part 16: Theory Questions

What is a Menu-Driven Chatbot?

A chatbot that provides predefined options for users to select.

Why use a Menu-Driven Chatbot?

It makes interaction easier and reduces user input errors.

Which Python concepts are used?

- while loop
 - if-elif-else
 - input()
 - print()
 - Dictionary (optional)
-

Why is break used?

To stop the chatbot when the user selects the Exit option.

Where are Menu-Driven Chatbots used?

ATMs, hospitals, restaurants, colleges, banking systems, and customer support.

Part 17: Interview Questions

What is a Menu-Driven Chatbot?

A chatbot that allows users to interact by selecting predefined menu options.

What is the difference between a Rule-Based Chatbot and a Menu-Driven Chatbot?

A Rule-Based Chatbot accepts typed input and matches it with predefined rules, while a Menu-Driven Chatbot asks users to select from a list of options.

Why are Menu-Driven Chatbots suitable for beginners?

Because they are simple to build and users don't need to type complex questions.

Can a Menu-Driven Chatbot understand natural language?

No. It only processes predefined menu selections.

Give three real-world examples of Menu-Driven Chatbots.

- ATM Systems
 - College Enquiry Bots
 - Restaurant Ordering Systems
-

Tasks for Day 60

Theory

1. Define a Menu-Driven Chatbot.
 2. Explain how it works.
 3. List five advantages.
 4. List four limitations.
 5. Compare Menu-Driven and Rule-Based Chatbots.
-

Practical Task 1

Create a **College Enquiry Menu Chatbot** with the following options:

- Course Fee
 - Duration
 - Eligibility
 - Location
 - Contact
 - Exit
-

Practical Task 2

Add two more options:

- Hostel Information
 - Placement Information
-

Practical Task 3

Use a Python dictionary to store chatbot responses.

Practical Task 4

Handle invalid menu selections without crashing the chatbot.

Practical Task 5

Modify the chatbot to ask:

Would you like to continue? (yes/no)

If the user enters **no**, the chatbot should exit gracefully.